

Write a program to read two integer values and print true if both the numbers end with the same digit, otherwise print false. Example: If 698 and 768 are given, program should print true as they both end with 8. Sample Input 1 25 53 Sample Output 1 false Sample Input 2 27 77 Sample Output 2 true

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a,b;
5     scanf("%d %d",&a,&b);
6     if(a%10 == b%10)
7     {
8         printf("true");
9     }
10    else
11    {
12        printf("false");
13    }
14    return 0;
15 }
```

|   | Input | Expected | Got   |   |
|---|-------|----------|-------|---|
| ✓ | 25 53 | false    | false | ✓ |
| ✓ | 27 77 | true     | true  | ✓ |

Passed all tests! ✓

**Objective**

In this challenge, we're getting started with conditional statements.

**Task**

Given an integer, *n*, perform the following conditional actions:

- If *n* is odd, print Weird
- If *n* is even and in the inclusive range of 2 to 5, print **Not Weird**
- If *n* is even and in the inclusive range of 6 to 20, print **Weird**
- If *n* is even and greater than 20, print **Not Weird**

Complete the stub code provided in your editor to print whether or not *n* is weird.

**Input Format**

A single line containing a positive integer, *n*.

**Constraints**

- $1 \leq n \leq 100$

**Output Format**

Print Weird if the number is weird; otherwise, print Not Weird.

**Sample Input 0**

3

**Sample Output 0**

Weird

**Sample Input 1**

24

**Sample Output 1**

Not Weird

**Explanation**

Sample Case 0: *n* = 3

*n* is odd and odd numbers are weird, so we print **Weird**.

Sample Case 1:  $n = 24$

$n > 20$  and  $n$  is even, so it isn't weird. Thus, we print **Not Weird**.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d\n",&n);
6     if(n%2==0)
7     {
8         if(n>=2 && n<=5)
9         {
10             printf("Not Weird");
11         }
12         if(n>=6 && n<=20)
13         {
14             printf("Weird");
15         }
16         if(n>20)
17         {
18             printf("Not Weird");
19         }
20     }
21     else
22     {
23         printf("Weird");
24     }
25 }
26 }
```

|   | Input | Expected  | Got       |   |
|---|-------|-----------|-----------|---|
| ✓ | 3     | Weird     | Weird     | ✓ |
| ✓ | 24    | Not Weird | Not Weird | ✓ |

Passed all tests! ✓

Three numbers form a Pythagorean triple if the sum of squares of two numbers is equal to the square of the third. For example, 3, 5 and 4 form a Pythagorean triple, since  $3^2 + 4^2 = 25 = 5^2$ . You are given three integers, a, b, and c. They need not be given in increasing order. If they form a Pythagorean triple, then print "yes", otherwise, print "no". Please note that the output message is in small letters. Sample Input 1 3 5 4 Sample Output 1 yes Sample Input 2 5 8 2 Sample Output 2 no

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a,b,c;
5     scanf("%d%d%d",&a,&b,&c);
6     if(a*a+b*b == c*c)
7     {
8         printf("yes");
9     }
10    else if(a*a+c*c == b*b)
11    {
12        printf("yes");
13    }
14    else if(b*b+c*c == a*a)
15    {
16        printf("yes");
17    }
18    else{
19        printf("no");
20    }
21    return 0;
22 }
```

|   | Input       | Expected | Got |   |
|---|-------------|----------|-----|---|
| ✓ | 3<br>5<br>4 | yes      | yes | ✓ |
| ✓ | 5<br>8<br>2 | no       | no  | ✓ |

Passed all tests! ✓

Write a program that determines the name of a shape from its number of sides. Read the number of sides from the user and then report the appropriate name as part of a meaningful message. Your program should support shapes with anywhere from 3 up to (and including) 10 sides. If a number of sides outside of this range is entered then your program should display an appropriate error message.

Sample Input 1

3

Sample Output 1

Triangle

Sample Input 2

7

Sample Output 2

Heptagon

Sample Input 3

11

Sample Output 3

The number of sides is not supported.

answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d", &n);
6     if (n==3)
7     {
8         printf("Triangle");
9     }
10
11     else if (n==4)
12     {
13         printf("Square");
14     }
15
16     else if (n==5)
17     {
18         printf("pentagon");
19     }
20
21     else if (n==6)
22     {
23         printf("Hexogon");
24     }
25
26     else if (n==7)
27     {
28         printf("Heptagon");
29     }
30
31     else if (n==8)
32     {
33         printf("Octagon");
34     }
35
36     else if (n==9)
37     {
38         printf("Nonagon");
39     }
40
41     else if (n==10)
42     {
43         printf("Decogon");
44     }
45
46     else
47     {
48         printf("The number of sides is not supported.");
49     }
50 }
```

|   | Input | Expected                              | Got                                   |   |
|---|-------|---------------------------------------|---------------------------------------|---|
| ✓ | 3     | Triangle                              | Triangle                              | ✓ |
| ✓ | 7     | Heptagon                              | Heptagon                              | ✓ |
| ✓ | 11    | The number of sides is not supported. | The number of sides is not supported. | ✓ |

Your code failed one or more hidden tests.

Your code must pass all tests to earn any marks. Try again.

The Chinese zodiac assigns animals to years in a 12-year cycle. One 12-year cycle is shown in the table below. The pattern repeats from there, with 2012 being another year of the Dragon, and 1999 being another year of the Hare.

| Year | Animal  |
|------|---------|
| 2000 | Dragon  |
| 2001 | Snake   |
| 2002 | Horse   |
| 2003 | Sheep   |
| 2004 | Monkey  |
| 2005 | Rooster |
| 2006 | Dog     |
| 2007 | Pig     |
| 2008 | Rat     |
| 2009 | Ox      |
| 2010 | Tiger   |
| 2011 | Hare    |

Write a program that reads a year from the user and displays the animal associated with that year. Your program should work correctly for any year greater than or equal to zero, not just the ones listed in the table.

Sample Input 1

2004

Sample Output 1

Monkey

Sample Input 2

2010

Sample Output 2

Tiger

```
1 #include<stdio.h>
2 int main()
3 {
4     int year;
5     scanf("%d",&year);
6     if (year %12==8)
7     {
8         printf("Dragon");
9     }
10 }
11 else if (year % 12==9)
12 {
13     printf("Snake");
14 }
15 else if (year%12==10)
16 {
17     printf("Horse");
18 }
19 else if (year%12==11)
20 {
21     printf("Sheep");
22 }
23 else if(year%12==0)
24 {
25     printf("Monkey");
26 }
27 else if(year%12==1)
28 {
29     printf("Rooter");
30 }
31 else if (year%12==2)
32 {
33     printf("Dog");
34 }
35 }
36 else if(year%12==3)
37 {
38     printf("Pig");
39 }
40 else if(year%12==4)
41 {
42     printf("Rat");
43 }
44 else if(year%12==5)
45 {
46     printf("Ox");
47 }
48 else if(year%12==6)
49 {
50     printf("Tiger");
51 }
52 else
```

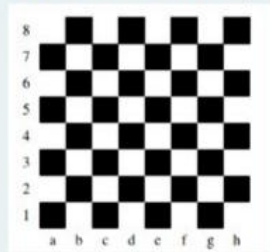


```
48 } else if(year%12==6)
49 {
50     printf("Tiger");
51 }
52 else
53 {
54     printf("Hare");
55 }
56 }
```

|   | Input | Expected | Got    |   |
|---|-------|----------|--------|---|
| ✓ | 2004  | Monkey   | Monkey | ✓ |
| ✓ | 2010  | Tiger    | Tiger  | ✓ |

Passed all tests! ✓

Positions on a chess board are identified by a letter and a number. The letter identifies the column, while the number identifies the row, as shown below:



Write a program that reads a position from the user. Use an if statement to determine if the column begins with a black square or a white square. Then use modular arithmetic to report the color of the square in that row. For example, if the user enters a1 then your program should report that the square is black. If the user enters d5 then your program should report that the square is white. Your program may assume that a valid position will always be entered. It does not need to perform any error checking.

Sample Input 1

a 1

Sample Output 1

The square is black.

Sample Input 2

d 5

Sample Output 2

The square is white.

Answer: (penalty regime: 0 %)

```

2 int main()
3 {
4     int num, sum;
5     char alpha;
6     scanf("%c%d", &alpha, &num);
7     sum = alpha + num;
8     if(sum%2==0)
9     {
10         printf("The square is black.");
11     }
12 }
13 else
14 {
15     printf("The square is white.");
16 }
17 }

```

|   | Input | Expected             | Got                  |   |
|---|-------|----------------------|----------------------|---|
| ✓ | a 1   | The square is black. | The square is black. | ✓ |
| ✓ | d 5   | The square is white. | The square is white. | ✓ |

Passed all tests! ✓

Question **1**

Correct

Marked out of  
3.00

Flag  
question

Some data sets specify dates using the year and day of year rather than the year, month, and day of month. The day of year (DOY) is the sequential day number starting with day 1 on January 1st.

There are two calendars - one for normal years with 365 days, and one for leap years with 366 days. Leap years are divisible by 4. Centuries, like 1900, are not leap years unless they are divisible by 400. So, 2000 was a leap year.

To find the day of year number for a standard date, scan down the Jan column to find the day of month, then scan across to the appropriate month column and read the day of year number. Reverse the process to find the standard date for a given day of year.

Write a program to print the Day of Year of a given date, month and year.

Sample Input 1

18

6

2020

Sample Output 1

170

```

1 #include<stdio.h>
2 int main()
3 {
4     int d,m,y,feb;
5     scanf("%d%d%d",&d,&m,&y);
6     if((y%100==0&& y%400) || (y%4!=0))
7         feb=29;
8     else
9         feb=28;
10    switch(m)
11    {
12        case 1:
13            printf("%d",d);
14            break;
15        case 2:
16            printf("%d",31+d);
17            break;
18        case 3:
19            printf("%d",31+feb+d);break;
20        case 4:
21            printf("%d",31+feb+31+d);
22            break;
23        case 5:
24            printf("%d",31+feb+feb+31+30+d);
25            break;
26        case 6:
27            printf("%d",31+feb+31+30+31+d);
28            break;
29        case 7:
30            printf("%d",31+feb+31+30+31+30+d);
31            break;
32        case 8:
33            printf("%d",31+feb+31+30+31+30+31+d);
34            break;
35        case 9:
36            printf("%d",31+feb+31+30+31+30+31+30+d);
37            break;
38        case 10:
39            printf("%d",31+feb+31+30+31+30+31+30+31+d);
40            break;
41        case 11:
42            printf("%d",31+feb+31+30+31+30+31+30+31+30+d);
43            break;
44        case 12:
45            printf("%d",31+feb+31+30+31+30+31+30+31+30+31+d);
46            break;
47    }
48 }

```

| Input | Expected | Got |   |
|-------|----------|-----|---|
| 18    | 170      | 170 | ✓ |
| 6     |          |     |   |
| 2020  |          |     |   |

Suppandi is trying to take part in the local village math quiz. In the first round, he is asked about shapes and areas. Suppandi is confused, he was never any good at math. And also, he is bad at remembering the names of shapes. Instead, you will be helping him calculate the area of shapes.

- When he says rectangle he is actually referring to a square.
- When he says square, he is actually referring to a triangle.
- When he says triangle he is referring to a rectangle
- And when he is confused, he just says something random. At this point, all you can do is say 0.

Help Suppandi by printing the correct answer in an integer.

Input Format

- Name of shape (always in upper case R à Rectangle, S à Square, T à Triangle)
- Length of 1 side
- Length of other side

Note: In case of triangle, you can consider the sides as height and length of base

Output Format

- Print the area of the shape.

Sample Input 1

T  
10  
20

Sample Output 1

200

Sample Input 2

S  
30  
40

Sample Output 2

600

Sample Input 3

R  
10

10

Sample Output 3

100

Sample Input 4

G

8

8

Sample Output 4

0

Sample Input

C

9

10

Sample Output 4

0

Explanation:

- First is output of area of rectangle
- Then, output of area of triangle
- Then output of area square
- Finally, something random, so we print 0

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a,b;
5     char ch;
6     scanf("%c%d%d",&ch,&a,&b);
7     switch(ch)
8     {
9         case 'R':
10            printf("%d", a*b);
11            break;
12         case 'S':
13            printf("%.0f", (0.5)*a*b);
14            break;
15         case 'T':
16            printf("%d",a*b);
17            break;
18         default:
19            printf("0");
20     }
21 }
```

|   | Input         | Expected | Got  |   |
|---|---------------|----------|------|---|
| ✓ | 1<br>10<br>20 | 200      | 200  | ✓ |
| ✓ | 5<br>30<br>40 | 600      | 600  | ✓ |
| ✓ | 8<br>2<br>11  | 0        | 0    | ✓ |
| ✓ | R<br>10<br>30 | 300      | 300  | ✓ |
| ✓ | 5<br>40<br>50 | 1000     | 1000 | ✓ |

Passed all tests! ✓



Superman is planning a journey to his home planet. It is very important for him to know which day he arrives there. They don't follow the 7-day week like us. Instead, they follow a 10-day week with the following days: Day Number Name of Day 1 Sunday 2 Monday 3 Tuesday 4 Wednesday 5 Thursday 6 Friday 7 Saturday 8 Kryptonday 9 Coluday 10 Daxamday Here are the rules of the calendar: - The calendar starts with Sunday always. - It has only 296 days. After the 296th day, it goes back to Sunday. You begin your journey on a Sunday and will reach after n. You have to tell on which day you will arrive when you reach there.

Input format: -

Contain a number n (0 < n)

Output format: Print the name of the day you are arriving on

Example Input

7

Example Output

Kryptonday

Example Input

1

Example Output Monday

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n, day;
5     scanf("%d",&n);
6     if(n<296)
7         day=n;
8     else
9         day=n-296;
10    day%=10;
11    day=day+1;
12    day%=10;
13    switch(day)
14    {
15        case 1:
16            printf("Sunday");
17            break;
18        case 2:
19            printf("Monday");
20            break;
21        case 3:
22            printf("Tuesday");
23            break;
24        case 4:
25            printf("Wednesday");
26            break;
27        case 5:
28            printf("Thursday");
29            break;
30        case 6:
31            printf("Friday");
32            break;
33        case 7:
34            printf("Saturday");
35            break;
36        case 8:
37            printf("Kryptonday");
38            break;
39        case 9:
40            printf("Coluday");
41            break;
42        case 10:
43            printf("Daxamday");
44            break;
45    }
46 }
```

|   | Input | Expected  | Got       |   |
|---|-------|-----------|-----------|---|
| ✓ | 7     | Kryptoday | Kryptoday | ✓ |
| ✓ | 1     | Monday    | Monday    | ✓ |

Passed all tests! ✓